On-Chip Design Verification with Xilinx FPGAs

Sreejeesh SG

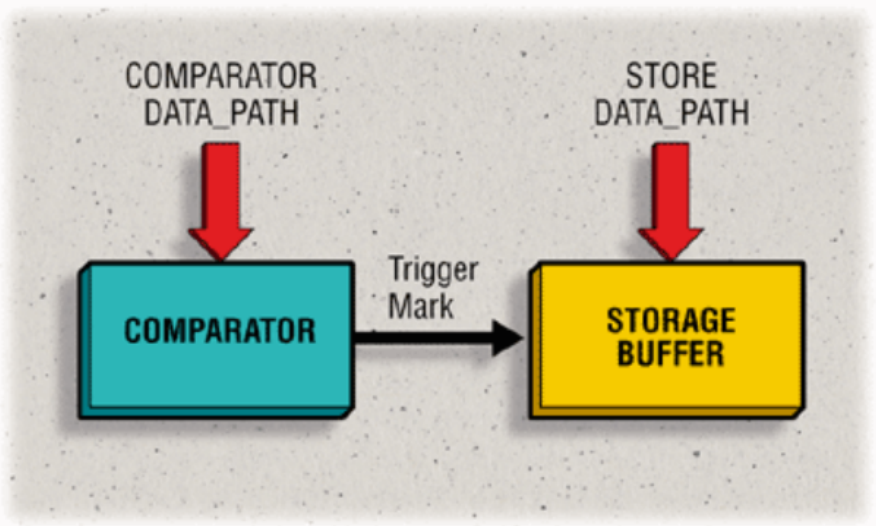
National Institute of Electronics and Information Technology-NIELIT

sreejeesh@nielit.gov.in

Controllability and Observability

- To address the controllability and observability issues of in-circuit verification Xilinx has created **ChipScope**. In ChipScope there are two fundamental intellectual property (IP) cores which give real-time observability and controllability.
- For observability ChipScope has a logic analyzer soft core called the integrated logic analyzer (ILA). For controllability ChipScope has the soft core called the virtual input-output core (VIO).
- You can examine and drive internal FPGA nodes deep in your design, all done real-time and in-circuit.

Design Observability



The ILA is a configurable logic analyser soft core that enables you to observe nodes inside your design. With the ILA you can inspect inputs and outputs of modules, sub-modules and even individual elements like flip flops and gates.

ILA

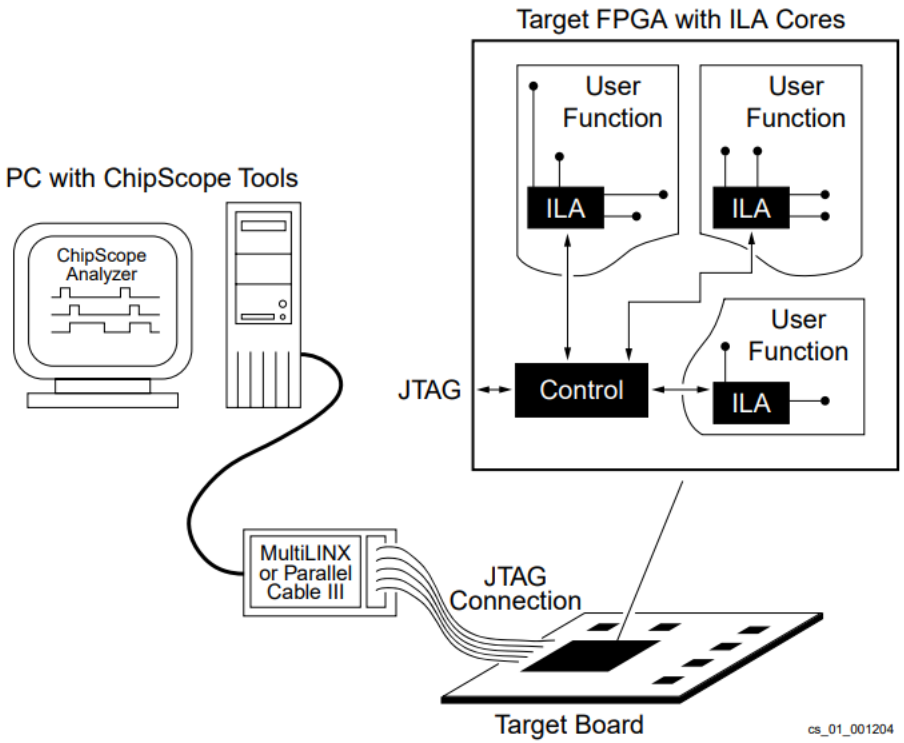


Figure 1: **ChipScope Block Diagram**

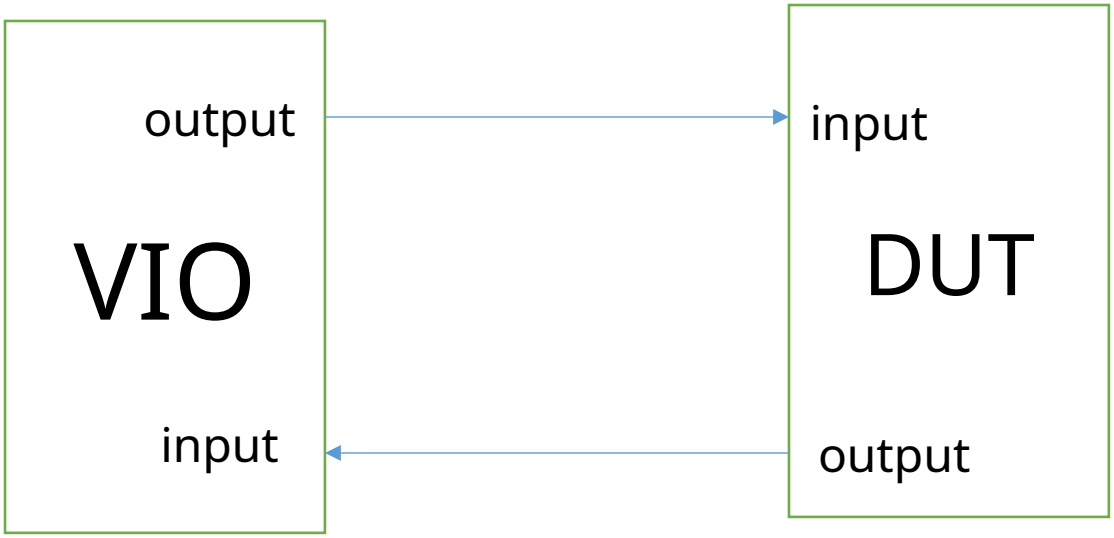
Integrated Logic Analyzer-ILA

- For the ILA, the comparator and storage buffer can probe the same or different nodes. When the comparator and storage buffer are both connected to the same nodes, the ILA behaves much like a typical bench top logic analyzer. In this case you set the comparator to detect a pattern seen in the storage buffer data path.
- When the comparator and storage buffer are connected to separate nodes, it gives you a new degree of freedom. With this feature you can have the ILA comparator attached to the address and control lines of an internal bus and have only the data bus connected to the storage buffer. In this way you can configure the comparator to search for certain addresses or ranges and store only the data bus samples.
- The ILA core gives you in-circuit observability but there are some issues to consider. The first is that the ILA requires you to supply a clock. Unlike a bench top logic analyzer that can work with its own sample clock, the ILA core cannot operate with out a user supplied clock. This restriction can work against a design that has tight timing margins, since the additional clock and probe connections may reduce the design performance.
- The second ILA issue to consider is the core size. The ILA unit that can consume the greatest amount of resources is the storage buffer. This is because the ILA uses internal block memory for the storage buffer block. This poses a problem for designs that already use the block memory or that require deep traces for in-circuit verification and debug.

Design Controllability

- For controllability Xilinx has created the virtual input output core (VIO). This core provides an alternate means of driving nodes inside the FPGA. Like the ILA the VIO can be connected to the inputs and outputs of modules, sub-modules and basic elements like flip flops.
- The difference is that the VIO core has specific ports for design inputs and outputs.

The basic structure of the VIO core



Virtual Input/Output (VIO)

- Monitor and drive internal FPGA signals in Real-time
- Customizable in size
- Detecting rising and falling transitions
- Interaction with core requires use of the Vivado Logic Analyzer feature

The Virtual Input/Output (VIO) core is a customizable core that can both monitor and drive internal FPGA signals in real time.

VIO

- The Virtual Input/Output (VIO) core is a customizable core that can both monitor and drive internal FPGA signals in real time. The number and width of the input and output ports are customizable in size to interface with the FPGA design. Because the VIO core is synchronous to the design being monitored and/or driven, all design clock constraints that are applied to your design are also applied to the components inside the VIO core. Run time interaction with this core requires the use of the Vivado® logic analyzer feature. The following figure is a block diagram of the new VIO core

constrain desired nets in your Verilog RTL to attach them to ILA

- You need to constrain desired nets in your Verilog RTL with: (* keep="true",mark_debug="true" *). The script will find these nets and attach them to an ILA, with the correct clock, trying to keep buses intact.

- (* keep="true",mark_debug="true" *) reg count [0:1023];

Demo is followed ….

Learn by doing it.

Thank You